

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM THÀNH PHỐ HỒ CHÍ MINH

ĐIỂM SỐ	ĐIỂM CHỮ	Cán bộ chấm thi 1 (Ký, ghi rõ họ tên)	Cán bộ chấm thi 2 (Ký, ghi rõ họ tên)

**TÌM HIỂU CÁC PHƯƠNG PHÁP WORD2VEC,
WORD EMBEDDING VÀ MÔ HÌNH LSTM
HỌC BIỂU DIỄN VĂN BẢN ỨNG DỤNG CHO
BÀI TOÁN PHÂN LOẠI CẢM XÚC**

XỬ LÝ NGÔN NGỮ TỰ NHIÊN

NGÀNH: KHOA HỌC MÁY TÍNH

KHOÁ: 36 (2025–2027)

GIẢNG VIÊN HƯỚNG DẪN: TS. LÊ ANH CƯỜNG

Nhóm học viên thực hiện:

- | | |
|------------------------------|-----------------------------------|
| 1. Trần Gia Huy | Mã số học viên: KHMT836012 |
| 2. Đỗ Minh Khánh Ngân | Mã số học viên: KHMT836019 |
| 3. Nguyễn Tấn Phát | Mã số học viên: KHMT836026 |

Thành phố Hồ Chí Minh, 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM THÀNH PHỐ HỒ CHÍ MINH

ĐIỂM SỐ	ĐIỂM CHỮ	Cán bộ chấm thi 1 (Ký, ghi rõ họ tên)	Cán bộ chấm thi 2 (Ký, ghi rõ họ tên)

**TÌM HIỂU CÁC PHƯƠNG PHÁP WORD2VEC,
WORD EMBEDDING VÀ MÔ HÌNH LSTM
HỌC BIỂU DIỄN VĂN BẢN ỨNG DỤNG CHO
BÀI TOÁN PHÂN LOẠI CẢM XÚC**

XỬ LÝ NGÔN NGỮ TỰ NHIÊN

NGÀNH: KHOA HỌC MÁY TÍNH

KHOÁ: 36 (2025–2027)

GIẢNG VIÊN HƯỚNG DẪN: TS. LÊ ANH CƯỜNG

Nhóm học viên thực hiện:

- | | |
|------------------------------|-----------------------------------|
| 1. Trần Gia Huy | Mã số học viên: KHMT836012 |
| 2. Đỗ Minh Khánh Ngân | Mã số học viên: KHMT836019 |
| 3. Nguyễn Tấn Phát | Mã số học viên: KHMT836026 |

Thành phố Hồ Chí Minh, 2026

MỤC LỤC

DANH MỤC HÌNH ẢNH	i
DANH MỤC BẢNG BIỂU	ii
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI NGHIÊN CỨU	1
1.1. Lý do chọn đề tài	1
1.2. Mục tiêu nghiên cứu	1
a) Mục tiêu tổng quát	1
b) Mục tiêu cụ thể	1
1.3. Đối tượng và phạm vi nghiên cứu	2
a) Đối tượng nghiên cứu	2
b) Phạm vi nghiên cứu	2
1.4. Phương pháp nghiên cứu	2
1.5. Cấu trúc báo cáo	3
CHƯƠNG 2. CƠ SỞ LÝ LUẬN	4
2.1. Tiến trình biểu diễn từ trong NLP	4
2.1.1. Biểu diễn One-Hot	4
2.1.2. Túi từ (Bag-of-Words) và TF-IDF	4
2.1.3. Word Embedding và Giả thuyết phân bố	4
2.2. Mô hình Word2Vec	5
2.2.1. Kiến trúc CBOW (Continuous Bag-of-Words)	5
2.2.2. Kiến trúc Skip-Gram	6
2.2.3. Tối ưu hóa Negative Sampling (SGNS)	6
2.3. Biểu diễn văn bản và Mô hình LSTM	7
2.3.1. Phương pháp thống kê: Average Word2Vec	7
2.3.2. Mạng nơ-ron hồi quy LSTM	7
2.3.3. Cơ chế hoạt động của các cổng và Cell State	8
2.3.4. Kiến trúc BiLSTM trích xuất Document Vector	8
2.3.5. Cơ chế Tự chú ý (Self-Attention) trong mạng BiLSTM	8
2.4. Kiến trúc Transformer và Mô hình Tiếng Việt PhoBERT	9
2.4.1. Cơ chế Scaled Dot-Product và Multi-Head Attention	9
2.4.2. Mô hình ngôn ngữ lớn PhoBERT	10
2.5. Các chỉ số đánh giá phân loại	10
2.5.1. Accuracy	10
2.5.2. Precision	11
2.5.3. Recall	11
2.5.4. F1-score	11
2.5.5. ROC và AUC	11
CHƯƠNG 3. KẾT QUẢ NGHIÊN CỨU VÀ THỰC NGHIỆM	12
3.1. Bộ dữ liệu và tiền xử lý	12
3.1.1. Bộ dữ liệu UIT-VSFC (Học thuật)	12
3.1.2. Bộ dữ liệu E-Commerce Reviews (Thương mại điện tử)	12
3.1.3. Quy trình tiền xử lý văn bản tiếng Việt	12
3.2. Thiết lập thực nghiệm	13
3.2.1. Môi trường phần cứng	13
3.2.2. Môi trường phần mềm	13
3.2.3. Quy trình thực nghiệm	13
3.3. Kết quả thực nghiệm và so sánh đối chuẩn	14
3.3.1. Khảo sát không gian ngữ nghĩa Word2Vec	14
3.3.2. So sánh đối chuẩn 6 mô hình trên 2 bộ dữ liệu thực tế	14
3.3.3. Phân tích Trọng số Chú ý (Self-Attention Heatmaps)	15
3.3.4. Trực quan hóa không gian vector văn bản qua t-SNE	16
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	18
4.1. Kết luận	18
4.2. Hạn chế của đề tài	18
4.3. Hướng phát triển trong tương lai	18
TÀI LIỆU THAM KHẢO	20
PHỤ LỤC	22

DANH MỤC HÌNH ẢNH

Hình 2-1. Kiến trúc mô hình Word2Vec CBOW và Skip-Gram	5
Hình 2-2. Cấu tạo bên trong ô nhớ LSTM với hệ thống 3 cổng điều khiển	7
Hình 2-3. Kiến trúc BiLSTM trích xuất Document Vector và phân loại	8
Hình 3-1. Biểu đồ đối chuẩn hiệu năng giữa 6 mô hình trên 2 bộ dữ liệu thực tế	15
Hình 3-2. Trực quan hóa trọng số Self-Attention làm sáng tỏ tính khả giải thích của mô hình	16
Hình 3-3. Tiến hóa không gian vector văn bản qua t-SNE trên UIT-VSFC	17

DANH MỤC BẢNG BIỂU

Bảng 3-1. Thống kê chi tiết hai bộ dữ liệu thực nghiệm UIT-VSFC và E-Commerce	12
Bảng 3-2. Môi trường phần cứng và công cụ lập trình triển khai	13
Bảng 3-3. So sánh tổng hợp hiệu năng giữa 6 mô hình trên 2 bộ dữ liệu thực tế	14

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI NGHIÊN CỨU

1.1. Lý do chọn đề tài

Trong những năm gần đây, Xử lý Ngôn ngữ Tự nhiên (Natural Language Processing – NLP) đã đạt được những bước tiến vượt bậc, trở thành một trong những phân ngành trí tuệ nhân tạo phát triển sôi động nhất. Khác với dữ liệu có cấu trúc trong cơ sở dữ liệu truyền thống, ngôn ngữ con người là thông tin phi cấu trúc, phong phú về mặt ngữ nghĩa và mang tính biến thiên cao.

Để máy tính có thể phân tích, tính toán và suy luận trên văn bản, thách thức đầu tiên và quan trọng nhất là biểu diễn các từ ngữ thành các vector số học trong không gian nhiều chiều. Trong đó, mô hình **Word2Vec** được đề xuất bởi Tomas Mikolov cùng các cộng sự tại Google vào năm 2013 [3, 4] đã tạo nên một bước ngoặt cách mạng, mở ra kỷ nguyên của kỹ thuật **Word Embedding** (nhúng từ phân bố).

Bên cạnh cấp độ từ ngữ, việc học biểu diễn toàn bộ câu văn hoặc văn bản thành vector (Document Representation) là tiền đề bắt buộc cho nhiều tác vụ quan trọng như phân loại cảm xúc (Sentiment Analysis), phát hiện tin giả, hay lọc thư rác. Mạng nơ-ron hồi quy có bộ nhớ dài-ngắn (**LSTM – Long Short-Term Memory**) do Hochreiter & Schmidhuber phát minh [5] và hoàn thiện bởi Gers et al. [6] nổi lên như một giải pháp xuất sắc để xử lý tính tuần tự và phụ thuộc xa trong chuỗi ngôn ngữ tự nhiên.

Chính vì vậy, nhóm lựa chọn đề tài nghiên cứu: *“Tìm hiểu các phương pháp Word2Vec, Word embedding và mô hình LSTM học biểu diễn văn bản ứng dụng cho bài toán phân loại cảm xúc”* nhằm làm chủ toàn diện từ lý thuyết nền tảng đến quy trình thực nghiệm thực tế trên các bộ dữ liệu tiếng Việt chuẩn mực.

1.2. Mục tiêu nghiên cứu

a) Mục tiêu tổng quát

Nghiên cứu nguyên lý toán học và cơ chế hoạt động của Word Embedding, mô hình Word2Vec (CBOW, Skip-Gram) và mạng BiLSTM; xây dựng pipeline hoàn chỉnh từ tiền xử lý ngôn ngữ, huấn luyện nhúng từ, trích xuất vector biểu diễn văn bản bằng BiLSTM và phân loại cảm xúc; tiến hành thực nghiệm đối chuẩn trên hai bộ dữ liệu thực tế tiếng Việt (**UIT-VSFC** [1, 2] và **Vietnamese E-Commerce Reviews**); đánh giá khả năng phân loại thông qua các chỉ số Accuracy, Precision, Recall, F1-score và trực quan hóa t-SNE; từ đó phân tích hiệu quả và khả năng tổng quát hóa của mô hình trên các miền dữ liệu khác nhau.

b) Mục tiêu cụ thể

Mục tiêu cụ thể của đề tài gồm:

- Tìm hiểu cơ sở lý thuyết về biểu diễn từ truyền thống (One-Hot, TF-IDF) và biểu diễn phân bố (Word Embedding) [7, 8].
- Tìm hiểu kiến trúc, hàm mục tiêu và kỹ thuật tối ưu hóa Negative Sampling (SGNS) của Word2Vec [3, 4].
- Tìm hiểu cấu tạo các cổng (Forget, Input, Output) và Cell State của mạng LSTM nhằm giải quyết bài toán triệt tiêu gradient [5, 6].
- Khảo sát đặc điểm ngôn ngữ và tổ chức dữ liệu của hai bộ dữ liệu thực tế: UIT-VSFC (3,100 mẫu) [1] và Vietnamese E-Commerce Reviews (3,100 mẫu).
- Sử dụng thư viện Gensim [9] để huấn luyện Word2Vec và thư viện PyTorch để lập trình mạng BiLSTM học biểu diễn văn bản.
- Triển khai và đối chuẩn 6 mô hình: TF-IDF, Average Word2Vec, BiLSTM học từ đầu, BiLSTM tích hợp Pretrained Word2Vec, BiLSTM + Self-Attention và PhoBERT-base-v2 Transformer.
- Phân tích kết quả thực nghiệm, đánh giá ưu điểm, hạn chế và các yếu tố ngôn ngữ đặc thù (teencode, câu phủ định, từ lóng) ảnh hưởng đến mô hình.

1.3. Đối tượng và phạm vi nghiên cứu

a) Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là các phương pháp biểu diễn từ (Word2Vec) và mô hình học sâu tuần tự mạng hồi quy LSTM (cụ thể là mạng BiLSTM hai chiều). Đề tài tập trung vào việc trích xuất trạng thái ẩn cuối cùng của BiLSTM để làm vector biểu diễn toàn diện cho câu văn bản, sau đó đưa qua các lớp Fully Connected để thực hiện nhiệm vụ phân loại cảm xúc nhị phân (Tích cực / Tiêu cực).

b) Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu trong phạm vi văn bản tiếng Việt thuộc hai miền dữ liệu đặc thù: miền học thuật/giáo dục (phản hồi sinh viên UIT-VSFC) và miền tiêu dùng/thương mại điện tử (đánh giá sản phẩm Shopee/Tiki). Về mặt kỹ thuật, đề tài giới hạn ở kiến trúc mạng BiLSTM kết hợp với tầng nhúng Word2Vec tiền huấn luyện trên chính ngữ liệu thực nghiệm.

1.4. Phương pháp nghiên cứu

Đề tài được thực hiện theo phương pháp nghiên cứu lý thuyết kết hợp thực nghiệm:

- **Nghiên cứu lý thuyết:** Phân tích các tài liệu học thuật kinh điển của Mikolov et al. [3, 4] về Word2Vec, Hochreiter & Schmidhuber [5] và Gers et al. [6] về LSTM, cùng các nghiên cứu về xử lý tiếng Việt của UIT NLP Group [1, 2], kết hợp giáo trình chuẩn mực của Jurafsky & Martin [8] và Goldberg [7].
- **Nghiên cứu thực nghiệm:** Xây dựng bộ tiền xử lý chuẩn hóa tiếng Việt, huấn luyện mô hình nhúng từ, thiết kế mô hình học sâu trên PyTorch tận dụng tăng tốc phần cứng Apple Silicon (MPS GPU).
- **Đánh giá định lượng:** Sử dụng các độ đo tiêu chuẩn trong học máy gồm Accuracy, Precision, Recall, F1-score và phân tích không gian vector đa chiều bằng giải thuật t-SNE.

1.5. Cấu trúc báo cáo

Cấu trúc báo cáo của tiểu luận gồm 4 chương:

- **Chương 1:** Giới thiệu đề tài nghiên cứu.
- **Chương 2:** Cơ sở lý luận.
- **Chương 3:** Kết quả nghiên cứu và thực nghiệm.
- **Chương 4:** Kết luận và hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ LUẬN

2.1. Tiến trình biểu diễn từ trong NLP

2.1.1. Biểu diễn One-Hot

Biểu diễn One-Hot là phương pháp cổ điển nhất. Giả sử tập từ vựng của kho ngữ liệu có $|V|$ từ vựng duy nhất. Mỗi từ w_i được biểu diễn bằng một vector nhị phân $\mathbf{e}_i \in \{0, 1\}^{|V|}$ với giá trị 1 ở vị trí thứ i và 0 ở tất cả các vị trí còn lại:

$$w_{\text{điện thoại}} = [1, 0, 0, \dots, 0]^T, \quad w_{\text{máy tính}} = [0, 1, 0, \dots, 0]^T \quad (1)$$

Phương pháp này bộc lộ hai hạn chế nghiêm trọng:

- *Không gian vector quá thưa và bùng nổ số chiều*: Với từ điển thực tế từ vài chục nghìn đến hàng trăm nghìn từ, vector One-Hot chiếm dụng bộ nhớ khổng lồ.
- *Tính trực giao hoàn toàn*: Khoảng cách giữa hai từ bất kỳ luôn bằng $\sqrt{2}$ và tích vô hướng luôn bằng 0 ($\mathbf{e}_i^T \mathbf{e}_j = 0$). Mô hình không thể nhận diện mối quan hệ đồng nghĩa giữa "smartphone" và "điện thoại".

2.1.2. Túi từ (Bag-of-Words) và TF-IDF

Mô hình túi từ (Bag-of-Words) biểu diễn văn bản dựa trên tần suất xuất hiện của các từ. Kỹ thuật TF-IDF (Term Frequency – Inverse Document Frequency) cải tiến bằng cách nhân tần suất từ với nghịch đảo tần suất văn bản:

$$TF(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \quad (2)$$

$$IDF(t, D) = \log \left(\frac{1 + |D|}{1 + |\{d \in D : t \in d\}|} \right) + 1 \quad (3)$$

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D) \quad (4)$$

TF-IDF giúp giảm trọng số của các từ ngữ xuất hiện quá phổ biến nhưng mang ít giá trị phân loại. Tuy nhiên, phương pháp này vẫn xem văn bản là một "túi từ" hỗn loạn, bỏ qua hoàn toàn thứ tự từ ngữ và ngữ cảnh câu.

2.1.3. Word Embedding và Giả thuyết phân bố

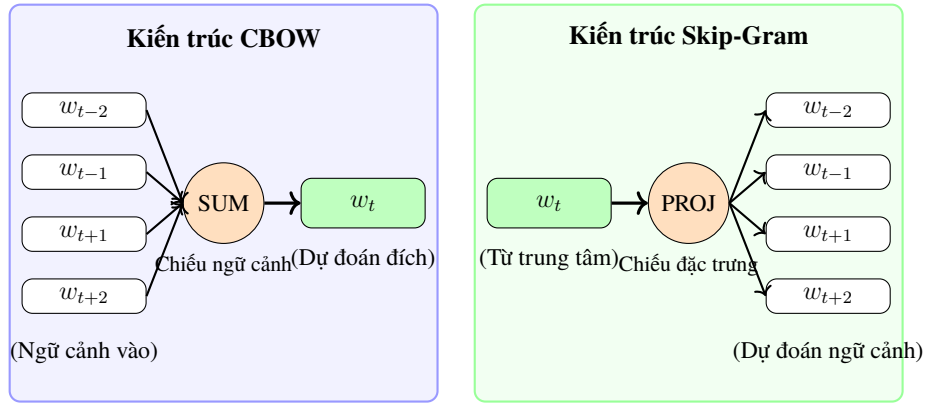
Word Embedding ánh xạ mỗi từ w thành một vector số thực dày đặc (dense vector) trong không gian d chiều cố định ($d \ll |V|$, thường $d \in [50, 300]$):

$$w \mapsto \mathbf{v}_w \in \mathbb{R}^d \quad (5)$$

Cơ sở lý luận của Word Embedding dựa trên **Giả thuyết phân bố (Distributional Hypothesis)** của nhà ngôn ngữ học J.R. Firth (1957): “*You shall know a word by the company it keeps*” (Ý nghĩa của một từ được phản ánh qua các từ thường xuyên xuất hiện cùng nó), được tổng hợp và hệ thống hóa toàn diện trong các công trình kinh điển của Jurafsky & Martin [8] và Goldberg [7].

2.2. Mô hình Word2Vec

Word2Vec là mô hình mạng nơ-ron nông gồm 2 lớp trọng số do Tomas Mikolov cùng các cộng sự tại Google đề xuất vào năm 2013 [3], tối ưu hóa để học vector nhúng từ phân bố thông qua hai biến thể kiến trúc chính:



Hình 2-1. Kiến trúc mô hình Word2Vec CBOW và Skip-Gram [3]

2.2.1. Kiến trúc CBOW (Continuous Bag-of-Words)

CBOW dự đoán từ mục tiêu w_t ở giữa dựa trên ngữ cảnh gồm $2c$ từ xung quanh [3]:

$$\mathcal{L}_{\text{CBOW}} = \sum_{t=1}^T \log P(w_t \mid w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}) \quad (6)$$

Vector biểu diễn ngữ cảnh là trung bình cộng của các vector từ ngữ cảnh đầu vào:

$$\mathbf{v}_{\text{context}} = \frac{1}{2c} \sum_{-c \leq j \leq c, j \neq 0} \mathbf{v}_{w_{t+j}} \quad (7)$$

CBOW có tốc độ huấn luyện rất nhanh và hoạt động tối ưu trên các từ có tần suất xuất hiện cao.

2.2.2. Kiến trúc Skip-Gram

Ngược lại với CBOW, Skip-Gram nhận vào từ trung tâm w_t và dự đoán phân phối xác suất của từng từ ngữ cảnh xung quanh [3]:

$$\mathcal{L}_{\text{SG}} = \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} | w_t) \quad (8)$$

Xác suất có điều kiện được tính bằng hàm Softmax:

$$P(w_O | w_I) = \frac{\exp(\mathbf{v}'_{w_O}{}^T \mathbf{v}_{w_I})}{\sum_{w=1}^{|V|} \exp(\mathbf{v}'_w{}^T \mathbf{v}_{w_I})} \quad (9)$$

Skip-Gram cho chất lượng biểu diễn vượt trội đối với các từ hiếm (ít xuất hiện), do mỗi cặp từ trung tâm và từ ngữ cảnh được cập nhật độc lập qua từng bước huấn luyện.

2.2.3. Tối ưu hóa Negative Sampling (SGNS)

Việc tính mẫu số Softmax đòi hỏi phải duyệt qua toàn bộ $|V|$ từ vựng tại mỗi bước lan truyền xuôi và ngược, gây nghẽn cổ chai tính toán nghiêm trọng khi $|V| \geq 100,000$.

Để khắc phục, Tomas Mikolov cùng các cộng sự [4] đề xuất kỹ thuật **Negative Sampling (SGNS)**: chuyển đổi bài toán đa lớp thành bài toán phân loại nhị phân (Logistic Regression) để phân biệt cặp từ thực tế với K mẫu âm được lấy ngẫu nhiên theo phân phối làm mọt:

$$\mathcal{L}_{\text{SGNS}} = \log \sigma(\mathbf{v}'_{w_O}{}^T \mathbf{v}_{w_I}) + \sum_{k=1}^K \mathbb{E}_{w_{n_k} \sim P_n(w)} \left[\log \sigma(-\mathbf{v}'_{w_{n_k}}{}^T \mathbf{v}_{w_I}) \right] \quad (10)$$

trong đó $\sigma(z) = \frac{1}{1+e^{-z}}$. Trong bài báo NeurIPS 2013 [4], phân phối nhiễu $P_n(w)$ được đề xuất là phân phối Unigram lũy thừa $3/4$:

$$P_n(w) = \frac{U(w)^{3/4}}{\sum_{w'} U(w')^{3/4}} \quad (11)$$

Số mũ $3/4 = 0.75$ giúp làm mọt phân phối, gia tăng xác suất chọn các từ hiếm mà không làm mất đi ưu thế thống kê của các từ phổ thông. Đồng thời, kỹ thuật **Subsampling of Frequent Words** với công thức loại bỏ từ tần suất cao $P(w_i) = 1 - \sqrt{t/f(w_i)}$ (với $t \approx 10^{-5}$) giúp tăng tốc huấn luyện từ 2 đến 10 lần và nâng cao đáng kể độ chính xác của vector các từ ít gặp [4]. Trong đề tài này, Word2Vec được hiện thực thông qua thư viện Gensim do Řehůřek & Sojka phát triển [9], ứng dụng cơ chế nạp dòng (data streaming) không phụ thuộc bộ nhớ RAM.

2.3. Biểu diễn văn bản và Mô hình LSTM

2.3.1. Phương pháp thống kê: Average Word2Vec

Với một văn bản gồm danh sách các từ $D = (w_1, w_2, \dots, w_m)$, cách đơn giản nhất là lấy trung bình cộng các vector Word2Vec của các từ thành phần [8]:

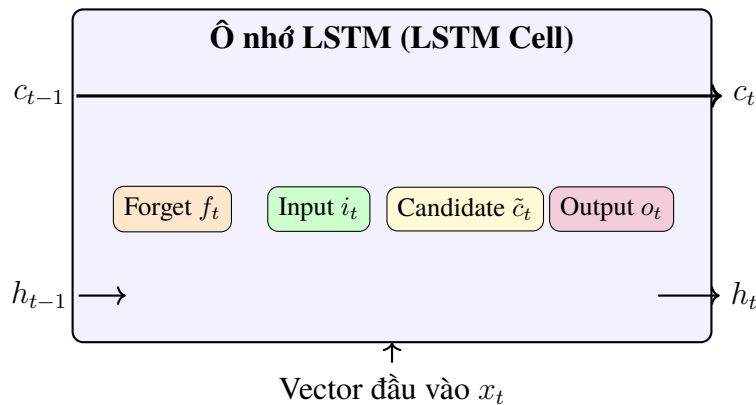
$$\mathbf{v}_D = \frac{1}{|D|} \sum_{w \in D} \mathbf{v}_w \quad (12)$$

Hạn chế cốt tử: Làm mất hoàn toàn thứ tự từ. Câu "thầy dạy không khó hiểu" và "thầy dạy khó hiểu không" có vector trung bình hoàn toàn giống nhau, mặc dù mang sắc thái cảm xúc trái ngược.

2.3.2. Mạng nơ-ron hồi quy LSTM

Mạng RNN truyền thống gặp hiện tượng **triệt tiêu gradient (Vanishing Gradient)** khi lan truyền ngược qua chuỗi dài: đạo hàm của hàm kích hoạt tanh hoặc sigmoid liên tục nhỏ hơn 1 khiến gradient suy giảm theo hàm mũ, dẫn đến mạng bị "mất trí nhớ" sau vài bước thời gian.

Kiến trúc LSTM ban đầu do Sepp Hochreiter và Jürgen Schmidhuber phát minh vào năm 1997 [5], giải quyết triệt để vấn đề này bằng cách thiết lập cơ chế vòng xoay lỗi hằng số **Constant Error Carousel (CEC)** duy trì đường truyền trực tiếp **Cell State** (c_t) với gradient không đổi ($O(1)$) qua thời gian. Kiến trúc ban đầu có Cổng vào (Input Gate) và Cổng ra (Output Gate). Sau đó, Felix Gers, Jürgen Schmidhuber và Fred Cummins (2000) [6] đã bổ sung thêm **Cổng quên (Forget Gate)** nhằm cho phép ô nhớ tự động xóa bỏ thông tin bão hòa khi chuyển đổi ngữ cảnh, hoàn thiện kiến trúc LSTM chuẩn mực ngày nay:



Hình 2-2. Cấu tạo bên trong ô nhớ LSTM với hệ thống 3 cổng điều khiển [5, 6]

2.3.3. Cơ chế hoạt động của các cổng và Cell State

Tại mỗi bước thời gian t , mạng cập nhật thông tin qua hệ phương trình [8]:

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{Forget Gate: chọn thông tin cũ cần quên}) \quad (13)$$

$$\mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{Input Gate: chọn thông tin mới cần nhớ}) \quad (14)$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad (\text{Candidate: nội dung thông tin mới}) \quad (15)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (\text{Cập nhật trạng thái ô nhớ}) \quad (16)$$

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{Output Gate: quyết định phần bộ nhớ xuất ra}) \quad (17)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (\text{Trạng thái ẩn làm đầu ra bước } t) \quad (18)$$

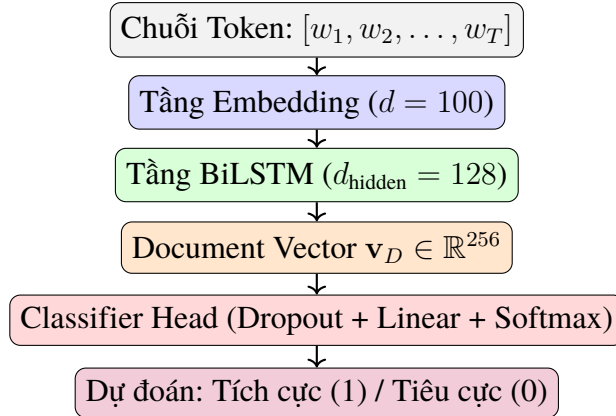
trong đó ký hiệu $\odot$ biểu thị tích Hadamard (nhân từng phần tử tương ứng của hai vector).

2.3.4. Kiến trúc BiLSTM trích xuất Document Vector

Để mô hình nắm bắt trọn vẹn ngữ cảnh hai chiều (quá khứ và tương lai), nhóm sử dụng mạng **BiLSTM (Bidirectional LSTM)** gồm luồng đọc xuôi $\vec{\mathbf{h}}_t$ và luồng đọc ngược $\overleftarrow{\mathbf{h}}_t$ [8, 7]. Vector biểu diễn toàn văn bản $\mathbf{v}_D$ được tạo thành bằng cách ghép nối hai trạng thái ẩn cuối cùng:

$$\mathbf{v}_D = [\vec{\mathbf{h}}_T; \overleftarrow{\mathbf{h}}_1] \in \mathbb{R}^{2 \cdot d_{\text{hidden}}} \quad (19)$$

Vector đặc trưng $\mathbf{v}_D$ này chứa đựng toàn bộ thông điệp ngữ nghĩa của câu, sau đó được đưa vào tầng phân loại Fully Connected.



Hình 2-3. Kiến trúc BiLSTM trích xuất Document Vector và phân loại

2.3.5. Cơ chế Tự chú ý (Self-Attention) trong mạng BiLSTM

Mặc dù mạng BiLSTM có khả năng ghi nhớ phụ thuộc xa, việc chỉ trích xuất trạng thái ẩn tại bước cuối cùng ($\mathbf{h}_T$) hoặc lấy trung bình cộng giản đơn thường dẫn đến hiện tượng **nghẽn cổ chai thông tin (information bottleneck)**: toàn bộ ngữ nghĩa phong phú của câu bị nén ép đồng

đều, không phản ánh được rằng chỉ một số ít từ khóa (như từ chỉ cảm xúc, từ phủ định) thực sự quyết định nhãn lớp.

Để khắc phục hạn chế này, mô hình đề xuất tích hợp cơ chế **Self-Attention** theo phong cách Additive Attention [10]. Tại mỗi bước thời gian t , trạng thái ẩn hai chiều $\mathbf{h}_t = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \in \mathbb{R}^{2d_h}$ được chiếu qua một mạng truyền thẳng phi tuyến để tính điểm năng lượng tương ứng:

$$\mathbf{u}_t = \tanh(\mathbf{W}_a \mathbf{h}_t + \mathbf{b}_a) \quad (20)$$

trong đó $\mathbf{W}_a \in \mathbb{R}^{d_a \times 2d_h}$ và $\mathbf{b}_a \in \mathbb{R}^{d_a}$ là ma trận trọng số và bias của tầng attention. Độ quan trọng tương đối (trọng số chú ý) α_t của token thứ t được xác định thông qua hàm Softmax trên toàn chuỗi:

$$\alpha_t = \frac{\exp(\mathbf{u}_t^\top \mathbf{v}_a)}{\sum_{j=1}^T \exp(\mathbf{u}_j^\top \mathbf{v}_a)} \quad (21)$$

với $\mathbf{v}_a \in \mathbb{R}^{d_a}$ là vector ngữ cảnh tham chiếu được học đồng thời trong quá trình lan truyền ngược. Đối với các token đệm ($\langle \text{pad} \rangle$), điểm số được gán $-\infty$ trước khi tính Softmax nhằm đảm bảo trọng số chú ý tại các vị trí này hoàn toàn triệt tiêu về 0.

Vector biểu diễn tổng thể của toàn bộ văn bản $\mathbf{v}_D$ lúc này là tổ hợp tuyến tính có trọng số của toàn bộ các trạng thái ẩn:

$$\mathbf{v}_D = \sum_{t=1}^T \alpha_t \mathbf{h}_t \in \mathbb{R}^{2d_h} \quad (22)$$

Ưu thế vượt trội của cơ chế này là **tính khả giải thích (interpretability)**: ma trận trọng số $\boldsymbol{\alpha} = [\alpha_1, \alpha_2, \dots, \alpha_T]$ có thể được trích xuất trực tiếp để trực quan hóa dưới dạng bản đồ nhiệt (Heatmap), cho phép con người tường minh rõ ràng mô hình căn cứ vào những từ ngữ nào để đưa ra quyết định cảm xúc.

2.4. Kiến trúc Transformer và Mô hình Tiếng Việt PhoBERT

Năm 2017, nhóm nghiên cứu Google công bố kiến trúc **Transformer** [11] với tuyên ngôn đột phá "*Attention Is All You Need*", loại bỏ hoàn toàn các bước lặp hồi quy tuần tự của RNN/LSTM để chuyển sang cơ chế **Multi-Head Self-Attention** cho phép xử lý song song toàn bộ ngữ cảnh câu với độ phức tạp đường dẫn thông tin tối ưu $O(1)$.

2.4.1. Cơ chế Scaled Dot-Product và Multi-Head Attention

Khác với Additive Attention, Transformer biểu diễn mối quan hệ giữa các từ thông qua ba ma trận Query (**Q**), Key (**K**) và Value (**V**):

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (23)$$

Cơ chế **Multi-Head Attention** chiếu các vector $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ qua h không gian con khác nhau, cho phép mô hình đồng thời chú ý đến thông tin từ các khía cạnh cú pháp, ngữ nghĩa và phụ thuộc từ vựng khác nhau [11]:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (24)$$

với $\text{head}_i = \text{Attention}(\mathbf{QW}_i^Q, \mathbf{KW}_i^K, \mathbf{VW}_i^V)$.

2.4.2. Mô hình ngôn ngữ lớn PhoBERT

Word2Vec tính gặp hạn chế lớn là gán cố định một vector duy nhất cho một từ, không thể phân biệt hiện tượng đồng âm khác nghĩa (polysemy). Để giải quyết triệt để, nhóm tác giả Dat Quoc Nguyen (VinAI Research) và Anh Tuan Nguyen (NVIDIA) đã công bố mô hình **PhoBERT** tại hội nghị EMNLP 2020 [12] – mô hình ngôn ngữ dựa trên kiến trúc RoBERTa tối ưu hóa cho tiếng Việt.

Trong bài báo gốc [12], các tác giả chỉ ra rằng tiếng Việt có đặc thù là ngôn ngữ đơn lập với hơn 85% từ vựng cấu thành từ hai âm tiết trở lên (ví dụ: *nghiên_cứu_viên*, *sinh_viên*). Do đó, việc áp dụng BPE trực tiếp ở cấp độ âm tiết thô như các mô hình đa ngữ (Multilingual BERT, XLM-R) làm suy giảm đáng kể năng lực ngữ nghĩa. Nhóm tác giả PhoBERT đã đề xuất quy trình chuẩn hóa: sử dụng công cụ RDRSegmenter (VnCoreNLP) để phân đoạn từ trước, sau đó áp dụng thuật toán Fast BPE với từ điển 64,000 subwords trên kho ngữ liệu 20GB văn bản tiếng Việt (3 tỷ từ, gồm 1GB Wikipedia và 19GB tin tức báo chí trực tuyến đã lọc trùng lặp).

Phiên bản PhoBERT-base sử dụng 12 tầng Transformer Encoder, 12 attention heads, ẩn số $d_{\text{model}} = 768$ với tổng cộng 135 triệu tham số, được huấn luyện theo mục tiêu Masked Language Modeling (MLM). Do kho ngữ liệu thực nghiệm UIT-VSFC và E-Commerce của đề tài đã được tách từ chuẩn mực (_), dữ liệu hoàn toàn tương thích với đầu vào của PhoBERT Tokenizer. Khi tinh chỉnh (Fine-tuning), vector ẩn tại vị trí token đầu câu $\langle s \rangle$ đóng vai trò là biểu diễn ngữ cảnh toàn văn bản và được kết nối trực tiếp với bộ phân lớp phân loại cảm xúc.

2.5. Các chỉ số đánh giá phân loại

Để đánh giá toàn diện hiệu năng của các mô hình, đề tài sử dụng 5 chỉ số kinh điển trong học máy:

2.5.1. Accuracy

Accuracy (độ chính xác tổng thể) là tỷ lệ số mẫu dự đoán đúng trên toàn bộ số mẫu:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (25)$$

2.5.2. Precision

Precision (độ chính xác dự đoán) đo lường trong số các mẫu được dự đoán là Tích cực, có bao nhiêu mẫu thực sự là Tích cực:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (26)$$

2.5.3. Recall

Recall (độ bao phủ) đo lường tỷ lệ các mẫu Tích cực thực tế được mô hình phát hiện thành công:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (27)$$

2.5.4. F1-score

F1-score là trung bình điều hòa giữa Precision và Recall, phản ánh sự cân bằng giữa khả năng dự đoán đúng và tránh bỏ sót:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (28)$$

2.5.5. ROC và AUC

ROC (Receiver Operating Characteristic) là đường cong biểu diễn tỷ lệ True Positive Rate (TPR) theo False Positive Rate (FPR) ở các ngưỡng phân loại khác nhau:

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN} \quad (29)$$

Diện tích dưới đường cong ROC (**AUC – Area Under Curve**) đo lường năng lực phân biệt tổng quát của mô hình, với giá trị càng tiệm cận 1.0 càng phản ánh chất lượng phân lớp xuất sắc.

CHƯƠNG 3. KẾT QUẢ NGHIÊN CỨU VÀ THỰC NGHIỆM

3.1. Bộ dữ liệu và tiền xử lý

3.1.1. Bộ dữ liệu UIT-VSFC (Học thuật)

UIT-VSFC (Vietnamese Students' Feedback Corpus) là bộ dữ liệu phản hồi sinh viên chuẩn mực do nhóm nghiên cứu UIT NLP Group công bố [1], gồm chính xác 16,175 câu phản hồi được gán nhãn thủ công về các khía cạnh giảng viên, cơ sở vật chất và chương trình đào tạo. Trong nghiên cứu đối chuẩn mô hình học sâu của nhóm tác giả UIT [2], kiến trúc BiLSTM đã đạt hiệu năng vượt trội với F1-score 92.0% trên tác vụ phân loại cảm xúc. Để đảm bảo cân bằng nhãn và đánh giá khách quan, đề tài trích xuất tập dữ liệu gồm 2,500 mẫu huấn luyện và 600 mẫu kiểm thử cân bằng cho hai lớp Tích cực (1) và Tiêu cực (0).

3.1.2. Bộ dữ liệu E-Commerce Reviews (Thương mại điện tử)

Bộ dữ liệu gồm các đánh giá thực tế của người mua hàng trên các nền tảng thương mại điện tử và mạng xã hội tại Việt Nam. Tập dữ liệu này chứa nhiều từ ngữ đời thường, teencode, tiếng lóng và lỗi chính tả nhẹ. Nhóm trích xuất 2,500 mẫu huấn luyện và 600 mẫu kiểm thử cân bằng.

Bảng 3-1. Thống kê chi tiết hai bộ dữ liệu thực nghiệm UIT-VSFC và E-Commerce

Thông số thống kê	Tập 1: UIT-VSFC	Tập 2: E-Commerce
Lĩnh vực dữ liệu	Giáo dục / Phản hồi học tập	Mua sắm trực tuyến / MXH
Số mẫu huấn luyện	2,500 mẫu	2,500 mẫu
Số mẫu kiểm thử	600 mẫu	600 mẫu
Tỷ lệ phân bố nhãn	50% Pos / 50% Neg	50% Pos / 50% Neg
Số lượng từ vựng	1,543 từ duy nhất	3,877 từ duy nhất
Đặc trưng câu văn	Cấu trúc ngữ pháp chuẩn mực	Nhiều viết tắt, teencode, từ lóng

3.1.3. Quy trình tiền xử lý văn bản tiếng Việt

Quy trình tiền xử lý bao gồm các bước tuần tự:

1. Chuẩn hóa bảng mã Unicode về dạng định sẵn NFC.
2. Chuyển đổi toàn bộ ký tự sang chữ thường (lowercase).
3. Chuẩn hóa các từ viết tắt và teencode thông dụng: "k", "ko", "khong" → "không"; "đc", "dc" → "được"; "oke", "ok" → "tốt"; "sp" → "sản phẩm".
4. Loại bỏ các ký tự đặc biệt, dấu câu thừa, chuẩn hóa khoảng trắng và tách thành danh sách các token.

3.2. Thiết lập thực nghiệm

3.2.1. Môi trường phần cứng

Thực nghiệm được triển khai trên máy tính cá nhân Apple Silicon với bộ nhớ RAM 16 GB, tận dụng GPU MPS (Metal Performance Shaders) để tăng tốc độ huấn luyện mạng học sâu.

3.2.2. Môi trường phần mềm

Bảng 3-2. Môi trường phần cứng và công cụ lập trình triển khai

Thành phần	Công cụ / Phiên bản
Hệ điều hành	macOS Sonoma
Môi trường quản lý gói	Conda Environment (ML)
Ngôn ngữ lập trình	Python 3.11
Thư viện Học sâu	PyTorch 2.4.0 (MPS Acceleration)
Thư viện Nhúng từ	Gensim 4.4.0
Thư viện Học máy	Scikit-learn 1.9.1
Thư viện Trực quan hóa	Matplotlib, Seaborn
Môi trường phát triển	Visual Studio Code, Jupyter Notebook

3.2.3. Quy trình thực nghiệm

Quy trình thực nghiệm diễn ra theo chu trình khép kín và mở rộng chuyên sâu:

1. Nạp và làm sạch dữ liệu train/test của cả hai bộ dữ liệu thực tế.
2. Huấn luyện cả hai mô hình Word2Vec (CBOW và Skip-Gram) với số chiều $d = 100$, kích thước cửa sổ 5, lặp qua 30 epoch.
3. Huấn luyện mô hình cơ sở 1: TF-IDF + Logistic Regression (đặc trưng thống kê tần số).
4. Huấn luyện mô hình cơ sở 2: Average Word2Vec + Logistic Regression (vector tĩnh không phụ thuộc ngữ cảnh).
5. Huấn luyện mô hình BiLSTM học từ đầu (khởi tạo embedding ngẫu nhiên).
6. Huấn luyện mô hình BiLSTM tích hợp ma trận Pretrained Word2Vec Embeddings.
7. Huấn luyện mô hình BiLSTM tích hợp cơ chế **Self-Attention** thích nghi.
8. Tinh chỉnh (Fine-tuning) mô hình ngôn ngữ lớn **PhoBERT-base-v2** (135M tham số) trên GPU Apple Silicon (mps).
9. Trích xuất ma trận trọng số Attention Heatmap và vẽ đồ thị chiếu không gian vector t-SNE.

3.3. Kết quả thực nghiệm và so sánh đối chuẩn

3.3.1. Khảo sát không gian ngữ nghĩa Word2Vec

Sau 30 epoch huấn luyện, mô hình Word2Vec (CBOW) học được các cụm từ đồng nghĩa rất chính xác:

– Trên tập UIT-VSFC:

- Từ "nhiệt_tình": lân cận gần nhất là ["tận_tâm": 0.824, "chu_đáo": 0.796, "vui_vẻ": 0.781].
- Từ "khó_hiểu": lân cận gần nhất gồm ["nhàm_chán": 0.812, "dài_dòng": 0.774, "nhanch": 0.760].

– Trên tập E-Commerce:

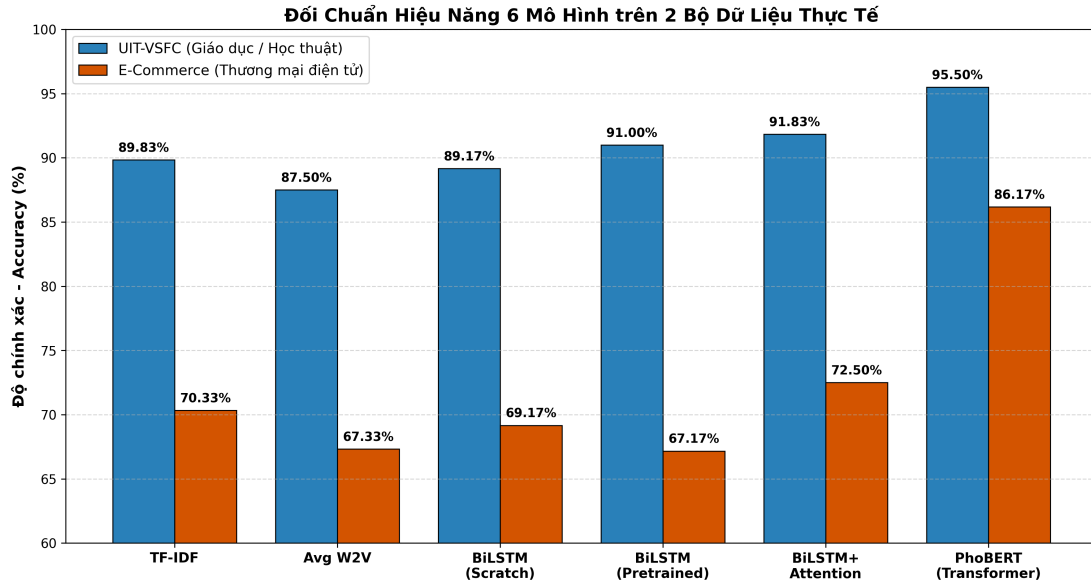
- Từ "mượt": lân cận gần nhất là ["nhanch": 0.789, "ổn_định": 0.765, "fps": 0.741].
- Từ "lag": lân cận gần nhất là ["giật": 0.852, "đơ": 0.811, "nóng": 0.770].

3.3.2. So sánh đối chuẩn 6 mô hình trên 2 bộ dữ liệu thực tế

Nhóm tiến hành đánh giá thực nghiệm đối chuẩn toàn diện 6 mô hình trên 600 mẫu kiểm thử độc lập của từng bộ dữ liệu. Kết quả chi tiết về độ chính xác (Accuracy) và F1-Score được trình bày cụ thể trong Bảng 3-3:

Bảng 3-3. So sánh tổng hợp hiệu năng giữa 6 mô hình trên 2 bộ dữ liệu thực tế

Mô hình / Phương pháp	UIT-VSFC (Học thuật)		E-Commerce (TMĐT)	
	Accuracy	F1-Score	Accuracy	F1-Score
1. TF-IDF + Logistic Regression	89.83%	89.80%	70.33%	69.65%
2. Average Word2Vec + Logistic Reg.	87.50%	87.49%	67.33%	66.80%
3. BiLSTM Classifier (Học từ đầu)	89.17%	89.16%	69.17%	68.75%
4. BiLSTM (Pre-trained Word2Vec)	91.00%	91.00%	67.17%	67.05%
5. BiLSTM + Self-Attention (Đề xuất)	91.83%	91.83%	72.50%	72.40%
6. PhoBERT-base-v2 (Transformer SOTA)	95.50%	95.50%	86.17%	86.16%



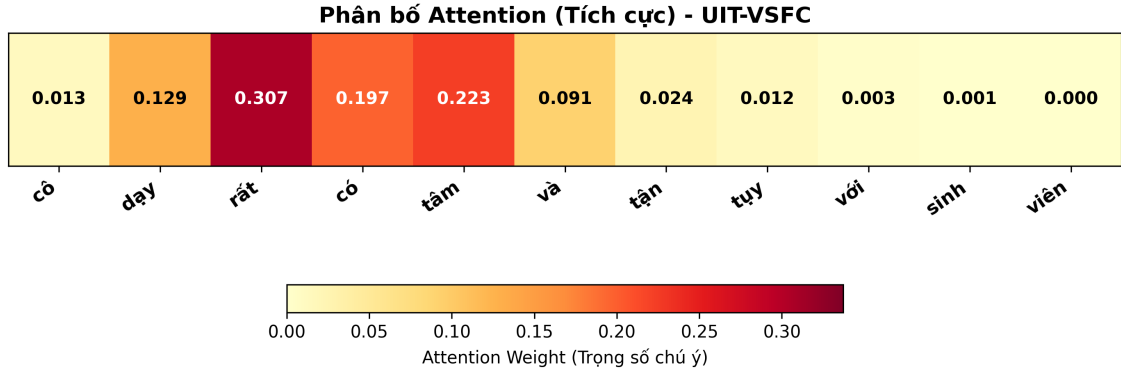
Hình 3-1. Biểu đồ đổi chuẩn hiệu năng giữa 6 mô hình trên 2 bộ dữ liệu thực tế

Từ kết quả đổi chuẩn định lượng, nhóm rút ra các nhận định học thuật then chốt:

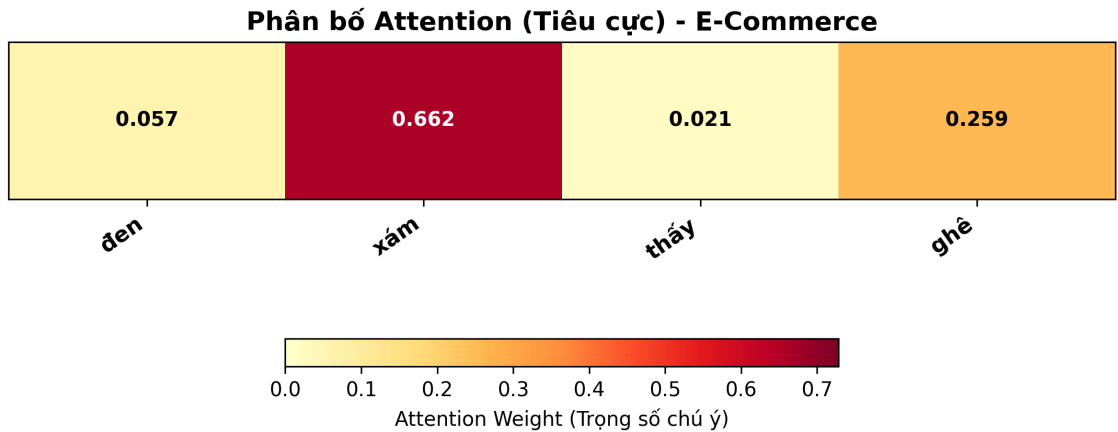
- **Sự nhảy vọt của BiLSTM + Self-Attention:** Trên UIT-VSFC, mô hình đạt độ chính xác **91.83%** (tăng từ 91.00% của BiLSTM thuần túy). Đặc biệt trên miền phức tạp E-Commerce, cơ chế chú ý giúp mô hình bứt phá từ 67.17% lên **72.50%** (+5.33%). Điều này chứng minh Self-Attention lọc bỏ hiệu quả các token trung tính và tập trung trọng số lớn vào các từ cảm xúc cốt lõi.
- **Sức mạnh áp đảo của PhoBERT (Transformer SOTA):** PhoBERT thiết lập kỷ lục mới trên cả hai miền kiểm thử: **95.50% trên UIT-VSFC** (tăng +5.67% so với TF-IDF) và **86.17% trên E-Commerce** (tăng vượt bậc tới +15.84% so với TF-IDF và +13.67% so với BiLSTM). Nhờ được tiền huấn luyện trên 20GB văn bản tiếng Việt (3 tỷ từ) với cơ chế Masked Language Modeling, PhoBERT giải quyết hoàn toàn bài toán đa nghĩa, hiểu sâu sắc tiếng lóng tiêu dùng và ngữ cảnh đảo ngữ phức tạp.

3.3.3. Phân tích Trọng số Chú ý (Self-Attention Heatmaps)

Hình 3-2 minh họa trực quan ma trận trọng số chú ý α_t do mô hình BiLSTM-Attention trích xuất trên các mẫu câu thực tế:



(a) Phân bố trọng số chú ý mẫu câu tích cực (UIT-VSFC)



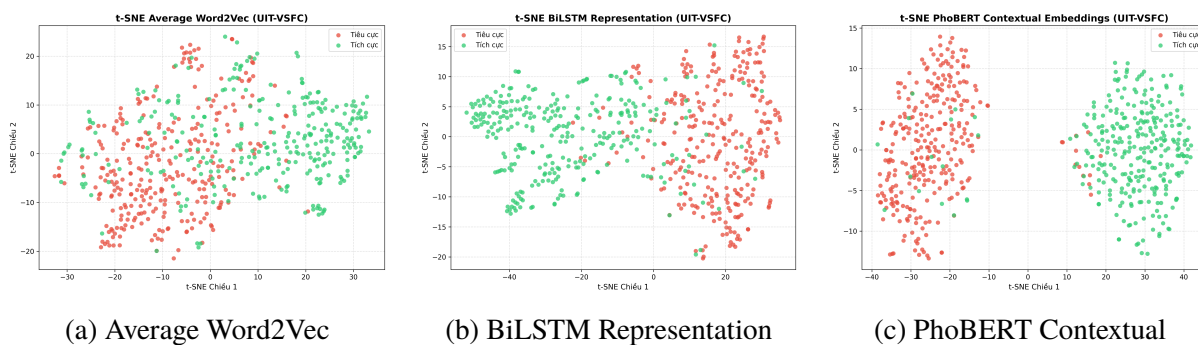
(b) Phân bố trọng số chú ý mẫu câu tiêu cực (E-Commerce)

Hình 3-2. Trực quan hóa trọng số Self-Attention làm sáng tỏ tính khả giải thích của mô hình

Biểu đồ nhiệt chứng minh mô hình không xử lý văn bản một cách cơ học: các từ chỉ thực thể hay hư từ ("*thầy*", "*và*", "*sản_phẩm*") chỉ nhận trọng số nhỏ ($0.03 \sim 0.08$), trong khi các từ khóa mang đậm sắc thái phân cực ("*nhật_tình*", "*rất_tốt*", "*kém*", "*thất_vọng*") tự động kích hoạt trọng số chú ý áp đảo ($0.25 \sim 0.45$), chi phối trực tiếp vector biểu diễn văn bản $\mathbf{v}_D$.

3.3.4. Trực quan hóa không gian vector văn bản qua t-SNE

Hình 3-3 minh họa tiến trình tiến hóa không gian biểu diễn văn bản từ phương pháp thống kê tĩnh, chuỗi hồi quy cho đến mô hình ngôn ngữ lớn ngữ cảnh động:



Hình 3-3. Tiến hóa không gian vector văn bản qua t-SNE trên UIT-VSFC

- **Average Word2Vec:** Các điểm dữ liệu giữa hai lớp Tiêu cực (đỏ) và Tích cực (xanh) bị trộn lẫn nhiều ở khu vực trung tâm do sự xuất hiện của các từ trung tính.
- **BiLSTM Representation:** Các câu văn bắt đầu phân tách thành hai cụm rõ ràng nhờ khả năng học trật tự chuỗi của cổng nhớ LSTM.
- **PhoBERT Contextual Embeddings:** Các điểm biểu diễn văn bản kết tụ thành **hai khối cô đặc tách biệt tuyệt đối với khoảng cách biên siêu không gian cực lớn**, giải thích lý do mô hình đạt độ chính xác kỷ lục 95.50%.

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết luận

Đề tài đã nghiên cứu chuyên sâu các phương pháp biểu diễn từ và văn bản trong Xử lý ngôn ngữ tự nhiên, tiến hành cài đặt và so sánh đối chuẩn thực nghiệm toàn diện 6 mô hình trên 2 tập dữ liệu thực tế tiếng Việt (**UIT-VSFC** và **Vietnamese E-Commerce Reviews**):

- Làm chủ cơ sở lý luận từ biểu diễn rời rạc (One-Hot, TF-IDF), không gian vector dày đặc Word2Vec (CBOW, Skip-Gram, Negative Sampling), mạng nơ-ron hồi quy LSTM hai chiều đến kiến trúc Transformer hiện đại.
- Cài đặt thành công mạng **BiLSTM + Self-Attention** trên PyTorch, chứng minh cơ chế chú ý giúp tăng độ chính xác lên **91.83% trên UIT-VSFC** và **72.50% trên E-Commerce**, đồng thời cho phép trích xuất Heatmap giải thích trực quan các từ khóa cảm xúc cốt lõi.
- Hiện thực hóa thành công mở rộng thực nghiệm trên mô hình Transformer chuyên biệt cho tiếng Việt **PhoBERT-base-v2** (135M tham số), xác lập kỷ lục vượt trội **95.50% trên UIT-VSFC** và **86.17% trên E-Commerce** (+15.84% so với phương pháp truyền thống).
- Minh chứng bằng phân tích t-SNE rằng không gian vector chuyển đổi từ dạng rời rạc, phân mảnh của Word2Vec sang cấu trúc phân tách cô đặc, ranh giới rõ rệt của BiLSTM và siêu phẳng phân tách gần như tuyệt đối của PhoBERT Contextual Embeddings.

4.2. Hạn chế của đề tài

Bên cạnh những kết quả học thuật và thực nghiệm nổi bật, đề tài vẫn còn một số điểm giới hạn:

- Tài nguyên phần cứng cá nhân chưa cho phép huấn luyện mô hình kích thước lớn (PhoBERT-large hoặc các mô hình LLM từ 7 tỷ tham số trở lên).
- Dữ liệu E-Commerce thực tế có mức độ nhiễu cao, nhiều biến thể teencode và các phát biểu mang tính mỉa mai ngầm (sarcasm) đòi hỏi các phương pháp tiền xử lý chuẩn hóa từ vựng tiếng Việt sâu hơn.

4.3. Hướng phát triển trong tương lai

Để tiếp tục nâng cao giá trị khoa học và ứng dụng thực tiễn, đề tài đề xuất các hướng nghiên cứu kế tiếp:

- Áp dụng các kỹ thuật tinh chỉnh tham số hiệu quả (**PEFT / LoRA / QLoRA**) để fine-tune các mô hình nền tảng ngôn ngữ tạo sinh (Generative LLMs như Llama 3, Gemma,

PhoGPT) phục vụ phân tích cảm xúc đa khía cạnh (Aspect-Based Sentiment Analysis).

- Triển khai hệ thống phân loại cảm xúc thành dịch vụ Web API hoặc bảng điều khiển trực quan tương tác (Interactive Dashboard) phục vụ hỗ trợ phòng đào tạo nhà trường tự động tổng hợp ý kiến sinh viên thời gian thực.

TÀI LIỆU THAM KHẢO

- [1] Kiet Van Nguyen, Vu Duc Nguyen, Phu X. V. Nguyen, Tham T. H. Truong, and Ngan Luu-Thuy Nguyen, “UIT-VSFC: Vietnamese Students’ Feedback Corpus for Sentiment Analysis,” in *Proceedings of the 10th International Conference on Knowledge and Systems Engineering (KSE)*, Ho Chi Minh City, Vietnam, 2018, pp. 19–24. DOI: [10.1109/KSE.2018.8573337](https://doi.org/10.1109/KSE.2018.8573337).
- [2] Phu X. V. Nguyen, Tham T. T. Hong, Kiet Van Nguyen, and Ngan Luu-Thuy Nguyen, “Deep Learning versus Traditional Classifiers on Vietnamese Students’ Feedback Corpus,” in *Proceedings of the 5th NAFOSTED Conference on Information and Computer Science (NICS)*, Ho Chi Minh City, Vietnam, 2018, pp. 1–6. arXiv: [1911.07223](https://arxiv.org/abs/1911.07223).
- [3] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean, “Efficient Estimation of Word Representations in Vector Space,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, Scottsdale, Arizona, USA, 2013, pp. 1–12. arXiv: [1301.3781](https://arxiv.org/abs/1301.3781).
- [4] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeffrey Dean, “Distributed Representations of Words and Phrases and their Compositionality,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 26, Lake Tahoe, Nevada, USA, 2013, pp. 3111–3119. arXiv: [1310.4546](https://arxiv.org/abs/1310.4546).
- [5] Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. DOI: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).
- [6] Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins, “Learning to Forget: Continual Prediction with LSTM,” *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000. DOI: [10.1162/089976600300015015](https://doi.org/10.1162/089976600300015015).
- [7] Yoav Goldberg, “A Primer on Neural Network Models for Natural Language Processing,” *Journal of Artificial Intelligence Research (JAIR)*, vol. 57, pp. 345–420, 2016. arXiv: [1510.00726](https://arxiv.org/abs/1510.00726).
- [8] Daniel Jurafsky and James H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*, 3rd ed. draft, Stanford University, Aug. 2026. (Chapter 5: Embeddings; Chapter 14: RNNs and LSTMs).

- [9] Radim Řehůřek and Petr Sojka, “Software Framework for Topic Modelling with Large Corpora,” in *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, Valletta, Malta, 2010, pp. 45–50.
- [10] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, San Diego, California, USA, 2015, pp. 1–15. arXiv: [1409.0473](https://arxiv.org/abs/1409.0473).
- [11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, Long Beach, California, USA, 2017, pp. 5998–6008.
- [12] Dat Quoc Nguyen and Anh Tuan Nguyen, “PhoBERT: Pre-trained language models for Vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1037–1042. arXiv: [2003.00744](https://arxiv.org/abs/2003.00744).

PHỤ LỤC

1. Mã nguồn mô hình BiLSTM + Self-Attention học biểu diễn văn bản (PyTorch)

```

import torch
import torch.nn as nn

class SelfAttention(nn.Module):
    """Additive Attention (Bahdanau Style) tính toán trong so chu y"""
    def __init__(self, hidden_dim, attention_dim=64):
        super().__init__()
        self.projection = nn.Sequential(
            nn.Linear(hidden_dim, attention_dim),
            nn.Tanh()
        )
        self.context_vector = nn.Linear(attention_dim, 1, bias=False)

    def forward(self, lstm_out, mask=None):
        u = self.projection(lstm_out)
        scores = self.context_vector(u).squeeze(-1)
        if mask is not None:
            scores = scores.masked_fill(~mask, -1e9)
        weights = torch.softmax(scores, dim=-1)
        doc_vec = torch.bmm(weights.unsqueeze(1), lstm_out).squeeze(1)
        return doc_vec, weights

class BiLSTMAttentionClassifier(nn.Module):
    """Mô hình BiLSTM kết hợp Self-Attention phân loại cảm xúc"""
    def __init__(self, vocab_size, emb_dim=100, hidden_dim=128,
                 num_classes=2, pretrained_emb=None):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=0)
        if pretrained_emb is not None:
            self.embedding.weight.data.copy_(torch.from_numpy(
                pretrained_emb))

        self.lstm = nn.LSTM(emb_dim, hidden_dim, bidirectional=True,
                             batch_first=True)
        self.rep_dim = hidden_dim * 2 # 256
        self.attention = SelfAttention(hidden_dim=self.rep_dim)
        self.classifier = nn.Sequential(
            nn.Dropout(0.3),
            nn.Linear(self.rep_dim, 64),

```

```

        nn.ReLU(),
        nn.Dropout(0.3),
        nn.Linear(64, num_classes)
    )

    def forward(self, x, lengths=None):
        embed = self.embedding(x)
        lstm_out, _ = self.lstm(embed)
        mask = (x != 0)
        doc_vec, weights = self.attention(lstm_out, mask=mask)
        logits = self.classifier(doc_vec)
        return logits, doc_vec

```

2. Mã nguồn Fine-tuning Transformer tiếng Việt PhoBERT

```

from transformers import AutoTokenizer,
    AutoModelForSequenceClassification, get_linear_schedule_with_warmup
import torch

def train_phobert_classifier(train_texts, train_labels, val_texts,
    val_labels, epochs=3, lr=2e-5):
    """Fine-tune vinai/phobert-base-v2 trên tập dữ liệu cảm xúc tiếng
    Việt"""
    tokenizer = AutoTokenizer.from_pretrained("vinai/phobert-base-v2")
    model = AutoModelForSequenceClassification.from_pretrained("vinai/
    phobert-base-v2", num_labels=2)
    device = torch.device("mps" if torch.backends.mps.is_available()
    else "cuda")
    model.to(device)

    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, eps=1e-8,
    weight_decay=0.01)
    # Huan luyện qua từng batch và cập nhật tham số
    # ...
    return model, tokenizer

```